Lab 4 - VibeCoding: Chat History

Student: Aadish Sarin (PES2UG24CS007) | Assigned repo: SETAPESU26/05_balloon_pop | Submission repo: github.com/aadishs-364/SE_Lab (Lab-4 folder) | AI tool: Claude (Claude Code)
Prompts used: 2 | Date: 16-10-2026

You

here is the pdf and the github repo assigned to me and help me provide the deliverables for it [Attached: Lab_4_VibeCoding_Student_handout.pdf] https://github.com/SETAPESU26/05_balloon_pop - my repo: https://github.com/aadishs-364/SE_Lab

Claude

I read the handout and the repo README. The handout asks you to fix the broken code, add every feature listed in the README, and upload the before/after evidence, the updated code and this chat history to a **Lab-4** folder in your own repo (no PR to SETAPESU26).

The README for **05_balloon_pop** lists one bug and three features:

• **Task 1** - click detection only works at the exact centre of a balloon
• **Task 2** - add normal, bonus and penalty balloon types with distinct colours
• **Task 3** - add 3 lives; an escaped balloon costs a life; game over at zero
• **Task 4** - add a 30-second timed round with a final-score screen and restart

I copied the game into **SE_Lab/Lab-4/code** and committed the untouched original first so the "before" state is preserved. *pygame* has no wheel for your Python 3.14, so I installed *pygame-ce*, a drop-in replacement (the code still does *import pygame*).

Task 1 - Fix the click-detection bug

Root cause: *check_pop* computes the *squared* distance from the click to the centre but compares it with the plain radius. For a radius-30 balloon that only accepts clicks within ~5.5 px of the centre. Comparing against radius² is equivalent to distance ≤ radius and avoids a square root.

```
diff --git a/Lab-4/code/balloon-pop/game/click_detection.py b/Lab-4/code/balloon-pop/game/click_detection.py
### game/click_detection.py
@@ -13,5 +13,6 @@ def check_pop(balloons, click_pos):
         dy = click_pos[1] - balloon.y
         distance_squared = dx * dx + dy * dy
-        if distance_squared <= balloon.radius:
+        # Compare squared distance with squared radius (same units), i.e. distance <= radius.
+        if distance_squared <= balloon.radius * balloon.radius:
             return balloon
     return None
```

Task 2 - Three balloon types

Each type carries its own points and colour: normal (pink) +10, bonus (gold) +30, penalty (dark grey) -20. Spawn odds are 70/15/15. Bonus and penalty balloons also get a +/- mark so they are distinguishable without relying on colour alone, and a legend in the top-right shows the scoring.

```
diff --git a/Lab-4/code/balloon-pop/game/balloon.py b/Lab-4/code/balloon-pop/game/balloon.py
### game/balloon.py
@@ -7,12 +7,20 @@ how click detection should work.
 import pygame

+# type name -> (points when popped, color)
+BALLOON_TYPES = {
+    "normal":  (10, (220, 90, 120)),   # pink
+    "bonus":   (30, (240, 190, 40)),   # gold
+    "penalty": (-20, (60, 60, 70)),    # dark grey
+}
+

 class Balloon:
-    def __init__(self, x, y, radius, speed, color=(220, 90, 120)):
+    def __init__(self, x, y, radius, speed, kind="normal"):
         self.x = x
         self.y = y
         self.radius = radius
         self.speed = speed
-        self.color = color
+        self.kind = kind
+        self.points, self.color = BALLOON_TYPES[kind]

     def update(self):
diff --git a/Lab-4/code/balloon-pop/game/game_engine.py b/Lab-4/code/balloon-pop/game/game_engine.py
### game/game_engine.py
@@ -9,10 +9,11 @@ lives system yet, no timer yet. Click detection also has a known bug
 import random

-from game.balloon import Balloon
+from game.balloon import Balloon, BALLOON_TYPES
 from game.click_detection import check_pop
 from game.renderer import WIDTH, HEIGHT

 SPAWN_INTERVAL_FRAMES = 45
-POINTS_PER_POP = 10
+# Spawn odds per balloon type: mostly normal, some bonus, some penalty.
+TYPE_WEIGHTS = {"normal": 70, "bonus": 15, "penalty": 15}


@@ -27,5 +28,6 @@ class GameEngine:
         x = random.randint(radius + 10, WIDTH - radius - 10)
         speed = random.uniform(1.5, 3.0)
-        self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed))
+        kind = random.choices(list(TYPE_WEIGHTS), weights=TYPE_WEIGHTS.values())[0]
+        self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed, kind=kind))

     def handle_click(self, pos):
@@ -33,5 +35,5 @@ class GameEngine:
         if popped is not None:
             self.balloons.remove(popped)
-            self.score += POINTS_PER_POP
+            self.score += popped.points   # normal +10, bonus +30, penalty -20

     def update(self):
@@ -50,2 +52,3 @@ class GameEngine:
         renderer.draw_scene(surface, self.balloons)
         renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
+        renderer.draw_legend(surface, font, BALLOON_TYPES)
diff --git a/Lab-4/code/balloon-pop/game/renderer.py b/Lab-4/code/balloon-pop/game/renderer.py
### game/renderer.py
@@ -17,4 +17,18 @@ def draw_scene(surface, balloons):
         pygame.draw.circle(surface, b.color, (int(b.x), int(b.y)), b.radius)
         pygame.draw.line(surface, (120, 120, 120), (b.x, b.y + b.radius), (b.x, b.y + b.radius + 12), 2)
+        # Mark special balloons with +/- so they're readable without relying on color alone.
+        if b.kind != "normal":
+            pygame.draw.line(surface, (255, 255, 255), (b.x - 6, b.y), (b.x + 6, b.y), 3)
+        if b.kind == "bonus":
+            pygame.draw.line(surface, (255, 255, 255), (b.x, b.y - 6), (b.x, b.y + 6), 3)

+
+
+def draw_legend(surface, font, types):
+    """Small key in the top-right showing each balloon color and its points."""
+    x = surface.get_width() - 150
+    for i, (kind, (points, color)) in enumerate(types.items()):
+        y = 14 + i * 24
+        pygame.draw.circle(surface, color, (x, y + 8), 8)
+        draw_text(surface, font, f"{kind} {points:+d}", (x + 14, y))
```

Task 3 - Lives / miss system

The player starts with 3 lives. Every balloon that falls past the bottom costs one life; popping never costs a life, whatever the type. At zero lives the game ends and clicks/spawning stop.

```
diff --git a/Lab-4/code/balloon-pop/game/game_engine.py b/Lab-4/code/balloon-pop/game/game_engine.py
### game/game_engine.py
@@ -16,4 +16,5 @@ SPAWN_INTERVAL_FRAMES = 45
 # Spawn odds per balloon type: mostly normal, some bonus, some penalty.
 TYPE_WEIGHTS = {"normal": 70, "bonus": 15, "penalty": 15}
+STARTING_LIVES = 3


@@ -23,4 +24,6 @@ class GameEngine:
         self.frames_until_spawn = 0
         self.score = 0
+        self.lives = STARTING_LIVES
+        self.game_over = False

     def _spawn_balloon(self):
@@ -32,10 +35,15 @@ class GameEngine:

     def handle_click(self, pos):
+        if self.game_over:
+            return
         popped = check_pop(self.balloons, pos)
         if popped is not None:
             self.balloons.remove(popped)
-            self.score += popped.points   # normal +10, bonus +30, penalty -20
+            self.score += popped.points   # normal +10, bonus +30, penalty -20; never costs a life

     def update(self):
+        if self.game_over:
+            return
+
         self.frames_until_spawn -= 1
         if self.frames_until_spawn <= 0:
@@ -46,5 +54,11 @@ class GameEngine:
             b.update()

-        self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)]
+        # Every balloon that escapes off the bottom costs one life.
+        escaped = [b for b in self.balloons if b.is_past_bottom(HEIGHT)]
+        if escaped:
+            self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)]
+            self.lives = max(0, self.lives - len(escaped))
+            if self.lives == 0:
+                self.game_over = True

     def draw(self, surface, font):
@@ -52,3 +66,7 @@ class GameEngine:
         renderer.draw_scene(surface, self.balloons)
         renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
+        renderer.draw_text(surface, font, f"Lives: {self.lives}/{STARTING_LIVES}", (10, 36), (200, 40, 60))
         renderer.draw_legend(surface, font, BALLOON_TYPES)
+
+        if self.game_over:
+            renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}")
```

Task 4 - 30-second timed round

The round counts down 30 s (frame-based at 60 FPS) and the timer turns red in the last 5 s. When time runs out or lives reach zero - whichever comes first - spawning and clicks stop, an end panel shows the reason and the final score, and pressing **R** resets score, lives and timer for a new round.

```
diff --git a/Lab-4/code/balloon-pop/game/game_engine.py b/Lab-4/code/balloon-pop/game/game_engine.py
### game/game_engine.py
@@ -9,4 +9,6 @@ lives system yet, no timer yet. Click detection also has a known bug
 import random

+import pygame
+
 from game.balloon import Balloon, BALLOON_TYPES
 from game.click_detection import check_pop
@@ -17,4 +19,6 @@ SPAWN_INTERVAL_FRAMES = 45
 TYPE_WEIGHTS = {"normal": 70, "bonus": 15, "penalty": 15}
 STARTING_LIVES = 3
+FPS = 60
+ROUND_FRAMES = 30 * FPS   # 30-second round, counted in frames (main loop runs at 60 FPS)


@@ -25,5 +29,12 @@ class GameEngine:
         self.score = 0
         self.lives = STARTING_LIVES
+        self.frames_left = ROUND_FRAMES
         self.game_over = False
+        self.end_reason = ""
+
+    def handle_keydown(self, key):
+        # R starts a fresh round: score, lives and timer all reset.
+        if self.game_over and key == pygame.K_r:
+            self.__init__()

     def _spawn_balloon(self):
@@ -46,4 +57,11 @@ class GameEngine:
             return

+        self.frames_left -= 1
+        if self.frames_left <= 0:
+            self.frames_left = 0
+            self.game_over = True
+            self.end_reason = "Time's up!"
+            return
+
         self.frames_until_spawn -= 1
         if self.frames_until_spawn <= 0:
@@ -61,4 +79,5 @@ class GameEngine:
             if self.lives == 0:
                 self.game_over = True
+                self.end_reason = "Out of lives!"

     def draw(self, surface, font):
@@ -67,6 +86,11 @@ class GameEngine:
         renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
         renderer.draw_text(surface, font, f"Lives: {self.lives}/{STARTING_LIVES}", (10, 36), (200, 40, 60))
+        seconds = -(-self.frames_left // FPS)   # ceil, so the display reads 30..1 then 0
+        renderer.draw_text(surface, font, f"Time: {seconds}s", (10, 62), (180, 40, 40) if seconds <= 5 else renderer.COLO
         renderer.draw_legend(surface, font, BALLOON_TYPES)

         if self.game_over:
-            renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}")
+            renderer.draw_banner(surface, font, [
+                f"{self.end_reason}  Final score: {self.score}",
+                "Press R to play a new round",
+            ])
diff --git a/Lab-4/code/balloon-pop/game/renderer.py b/Lab-4/code/balloon-pop/game/renderer.py
### game/renderer.py
@@ -37,6 +37,12 @@ def draw_text(surface, font, text, pos, color=COLOR_TEXT):


-def draw_banner(surface, font, text):
-    surf = font.render(text, True, (180, 40, 40))
-    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
-    surface.blit(surf, rect)
+def draw_banner(surface, font, lines):
```

```
+    """Centered panel with one or more lines of text (used for the end-of-round screen)."""
+    w, h = surface.get_width(), surface.get_height()
+    panel = pygame.Rect(0, 0, w - 120, 40 + 32 * len(lines))
+    panel.center = (w // 2, h // 2)
+    pygame.draw.rect(surface, (255, 255, 255), panel, border_radius=10)
+    pygame.draw.rect(surface, (180, 40, 40), panel, width=3, border_radius=10)
+    for i, line in enumerate(lines):
+        surf = font.render(line, True, (180, 40, 40))
+        surface.blit(surf, surf.get_rect(center=(w // 2, panel.top + 36 + 32 * i)))
diff --git a/Lab-4/code/balloon-pop/main.py b/Lab-4/code/balloon-pop/main.py
### main.py
@@ -5,4 +5,5 @@ Run with:  python3 main.py

 Click balloons to pop them before they reach the bottom.
+Pink = +10, gold = +30, grey = -20. 3 lives, 30-second round, R to restart.
 """

@@ -28,4 +29,6 @@ def main():
             elif event.type == pygame.MOUSEBUTTONDOWN:
                 engine.handle_click(event.pos)
+            elif event.type == pygame.KEYDOWN:
+                engine.handle_keydown(event.key)

         engine.update()
```

Verification

I added **test_game.py**, which checks each task: an edge click now pops (and a click outside does not), each type changes the score by the right amount without costing a life, each escaped balloon costs one life with game over at zero, and the round ends after 30 s, ignores clicks, and resets on R. It also renders a full round headlessly. Result: **All lab task checks passed.**

Everything is committed one task per commit and pushed to **aadishs-364/SE_Lab** under **Lab-4/**.

You

now take screenshots, as our mam told to do instead of video, before the code was updated and after the code was updated, and then provide me the chat history as well

Claude

I generated the screenshots from the real game code - the original version for "before" and the fixed version for "after" - and saved them in **Lab-4/screenshots/**. Each one has a caption strip underneath, and a red crosshair marks where a click was made. For the click bug, both versions use the identical setup (same balloon, same click point) so the before/after comparison is exact.

Screenshot	What it shows
before_1_gameplay.png	Original game: one balloon type, only a score - no lives, no timer
before_2a / 2b	Balloon of radius 40; click 30 px from its centre (inside it) - NOT popped, score stays 0
before_3_no_penalty.png	Many balloons escaped the bottom - nothing happens, the game never ends
after_1_gameplay.png	New HUD: score, lives, timer and the balloon-type legend
after_2a / 2b	Exact same click on the same balloon now pops it -> score 10
after_3_balloon_types.png	Normal (+10), bonus (+30, marked +), penalty (-20, marked -)
after_4_bonus_and_penalty_popped.png	Popped bonus then penalty -> 30 - 20 = 10, lives unchanged
after_5_lives_lost.png	A balloon escaped -> lives 2/3
after_6_game_over.png	Lives reached 0 -> game over panel with final score
after_7_times_up.png	Timer reached 0 during play -> "Time's up!" panel with final score
after_8_restart.png	Pressed R -> score 0, lives 3/3, timer back to 30 s

I also exported this conversation as **Lab-4/chat_history.pdf**. Final Lab-4 folder: *code/* (updated game), *screenshots/* (before and after), and *chat_history.pdf*.